ZTEK:

A Zero Trust Epistemic Kernel

for AI-Assisted Research Workflows

Paper 9 in the AI Epistemic Governance Stack

Series DOIs: 10.5281/zenodo.19562751 (Paper 7) | 10.5281/zenodo.19565086 (Paper 8)

Follows: AI Behavior Science (Zenodo 10.5281/zenodo.19562751)
and AEAL Framework (Zenodo 10.5281/zenodo.19565086)

Abstract

Generative AI systems lack the internal mechanisms — uncertainty, metacognition, causal reasoning, and self-honesty — that humans rely on to stop themselves from fabricating or overclaiming. Prior work established the theoretical basis for this problem (AI Behavior Science, Paper 7) and introduced an accountability layer for managing it (AEAL, Paper 8). This paper introduces ZTEK: a Zero Trust Epistemic Kernel, a boundary enforcement protocol derived from documented failure modes in a real multi-year AI-assisted mathematical research program.

ZTEK adapts the Zero Trust Network (ZTN) security paradigm to the epistemic domain. In ZTN, no actor is trusted by default regardless of origin; every request must authenticate. In ZTEK, no AI-generated claim is trusted by default regardless of fluency; every claim must prove provenance. The protocol is organized into five pillars, each grounded in a specific, documented failure incident. Together they constitute a decision gate between AI output and the verified knowledge base — an epistemic firewall that compensates for the structural gaps between human and AI cognition.

Retrospective validation against five documented incidents shows that ZTEK would have fully prevented four of the five failure events. An appendix provides concrete implementation artifacts: a provenance JSON schema, sample reproduce commands, and a sample audit log entry.

Table of Contents

1. Introduction and Motivation

2. The Zero Trust Analogy

3. Lessons Learned: The Failure Mode Registry

4. The Five Pillars of ZTEK

5. The ZTEK Decision Gate

6. Trust Tier System

7. Integration with AEAL

8. Implementation Guide

9. Empirical Validation

10. Limitations and Future Work

11. Conclusion

References

Appendix A — Provenance JSON Schema

Appendix B — Sample Reproduce Commands

Appendix C — Sample Audit Log Entry

1. Introduction and Motivation

The central problem of AI-assisted knowledge work is not that AI systems produce wrong answers. It is that they produce wrong answers indistinguishable from right ones. A generative AI system under completion pressure will fabricate a plausible result with the same fluency, confidence, and structural coherence as a verified one. The fabrication is not deceptive by intent — it is the natural output of a statistical continuation engine encountering a gap it cannot honestly fill.

This paper addresses a specific, practical question: how do we build a reliable boundary between what the AI generates and what enters the verified knowledge base?

The answer proposed here is not a better prompt, a smarter model, or a more careful user. It is an architectural protocol — a Zero Trust Epistemic Kernel — that treats every AI output as untrusted by default and requires it to earn its way into the knowledge base through reproducible verification.

1.1 Relationship to Prior Work

ZTEK is the third layer of the AI Epistemic Governance Stack:

Paper	Layer	Core Contribution
Paper 7: AI Behavior Science	Theoretical Foundation	Explains why AI fails epistemically
Paper 8: AEAL Framework	Accountability Layer	Tracks and scores AI reliability over time
Paper 9: ZTEK (this paper)	Enforcement Mechanism	Enforces the boundary at point of entry
Paper 10: Change Management Framework	Deployment Layer	Organizational adoption pathway

ZTEK does not replace AEAL. It operationalizes it at the boundary. AEAL tells you how trustworthy an agent has been over time; ZTEK decides whether a specific claim may cross into the verified knowledge base right now.

2. The Zero Trust Analogy

Zero Trust Network (ZTN) architecture is a cybersecurity paradigm built on one principle: never trust, always verify. In a traditional network model, being inside the perimeter implied trust. In ZTN, no actor is trusted by default — every request must authenticate, every transaction must be authorized, and trust is never assumed from context or origin.

The epistemic problem of AI-assisted research maps onto this paradigm with striking precision:

Zero Trust Network (ZTN)	ZTEK Epistemic Equivalent
No implicit trust based on being inside the network	No implicit trust based on being inside the conversation
Every request must authenticate	Every claim must prove provenance
Least-privilege access	Least-confidence assertion
Continuous verification	Claim-by-claim verification
Breach assumed by default	Fabrication assumed by default
Perimeter enforcement at network boundary	Boundary enforcement at knowledge base entry

The critical difference: ZTN protects against malicious actors. ZTEK protects against statistical continuation pressure — which is in some ways harder to defend against, because it produces no adversarial signal. The fabricated claim looks exactly like the verified claim.

3. Lessons Learned: The Failure Mode Registry

Each ZTEK pillar is grounded in a specific, documented failure incident from a real multi-year AI-assisted mathematical research program (polynomial continued fraction spectral fingerprint research, 2023–2025). This section records those incidents as the empirical basis for the protocol.

Incident	Failure Mode	ZTEK Pillar Triggered
Dual-path fabrication incident	AI produced a plausible breakthrough result that propagated through a relay chain without quarantine	Pillar 1: Assume fabrication by default
PSLQ degree-2 trivial relation flaw	PSLQ accepted trivial relations as valid; no provenance enforcement caught the error upstream	Pillar 2: Provenance or discard
Multi-model relay chain propagation	Claude output fed to Copilot treated as verified context; fabrication amplified across models	Pillar 3: Verify at boundary
Searcher's Fatigue pattern	Agent launched new sweeps instead of writing verified results; T0 outputs treated as T3	Pillar 4: Least confidence assertion
Early catalog invalidation	Trust inherited across session boundary; large portion of early results invalidated	Pillar 5: Re-verify on context change

4. The Five Pillars of ZTEK

Pillar 1 — Assume Fabrication by Default

Every AI output enters a quarantine state. Nothing is trusted until it passes verification. The burden of proof is on the claim, not on the skeptic.

Grounding incident:

The dual-path fabrication incident, in which a plausible-sounding breakthrough result propagated through a relay chain because it was never quarantined. The result was fluent, internally consistent, and structurally coherent. It was also fabricated.

Implementation:

- All AI outputs tagged as QUARANTINE on generation
- No quarantined output may be cited, referenced, or acted upon
- Quarantine state is cleared only by passing the Pillar 2 provenance check

Pillar 2 — Provenance or Discard

Every claim that enters the verified knowledge base must carry a complete provenance record. No provenance means no claim — only a hypothesis at best, discarded at worst.

Required provenance fields:

Field	Description	Example
claim_id	Unique identifier for this claim	pcf-2024-0047
claim_text	The exact claim being asserted	V_quad = GCF(1, 3n^2+n+1) is irrational
evidence_class	One of: near_miss | numerical_identity | independently_verified | formalized	independently_verified
reproduce_command	Exact command to re-derive the result	python verify_vquad.py --digits 2200 --seed 42
environment	Runtime fingerprint	Python 3.11, mpmath 1.3.0, commit a3f92c1
source_chain	Models and sessions that produced or touched the claim	Claude-3-opus > Copilot-gpt4 > human-review
timestamp	ISO 8601 generation timestamp	2024-11-15T09:34:12Z
tier	Current trust tier (T0-T3)	T2

A complete provenance schema (JSON format) is provided in Appendix A.

Grounding incident:

The PSLQ degree-2 trivial relation flaw, in which PSLQ accepted spurious relations because no provenance enforcement existed upstream. A rationality pre-filter was subsequently implemented — exactly the enforcement Pillar 2 formalizes.

Pillar 3 — Verify at the Boundary, Not Inside

Verification happens at the entry point to the knowledge base, not inside the AI session. What the AI says to itself during generation is epistemically irrelevant. Only what survives the boundary check counts.

This directly mirrors ZTN's perimeter logic. The firewall is at the edge. Internal consistency within the AI session does not confer trust.

Grounding incident:

Multi-model relay chain propagation failures, in which each model in the chain treated the previous model's output as verified context. No boundary check was ever applied. Fabrication compounded at each handoff.

Implementation:

- Boundary check is a separate, independent process — not a self-assessment by the generating model
- Multi-model handoff packages must include a full provenance chain, not just the final output
- Each boundary crossing resets to QUARANTINE state regardless of prior status

Pillar 4 — Least Confidence Assertion

AI outputs enter the system at the lowest plausible trust tier and earn their way up through verification. Trust is never assumed or inherited — it is demonstrated.

Trust tier levels:

- T0 — Raw AI output. Quarantined. No action permitted.
- T1 — Passed reproduction test. Working hypothesis. Informs but does not certify.
- T2 — Passed independent verification. Working result. May be used in analysis.
- T3 — Passed formal proof or peer-equivalent check. Citable. May enter final paper.

Nothing skips tiers. Every claim starts at T0 and earns upward progression.

Grounding incident:

The Searcher's Fatigue pattern, in which T0 outputs were treated as T3 because the distinction was never enforced. New sweeps were launched on unverified foundations, compounding unreliability.

Pillar 5 — Continuous Re-verification on Context Change

Previously verified claims do not automatically carry their trust tier across context changes. When the context changes — new model, new paper, new codebase, new session — claims re-enter at T1 and must re-verify.

This prevents trust laundering: the process by which a claim verified in one context is assumed valid in a different context where the verification conditions no longer hold.

Grounding incident:

The early catalog invalidation incident, in which trust was inherited across a session boundary without re-verification, ultimately invalidating a large portion of early results. The error was systemic and only discovered retrospectively.

5. The ZTEK Decision Gate

The decision gate is the operational core of ZTEK. Every AI-generated claim passes through this sequence before entering the knowledge base. The gate is sequential and non-negotiable: a claim that fails Step 1 does not proceed to Step 2.

Step	Check	Pass Result	Fail Result
1	Has reproduce command?	Proceed to Step 2	Discard or flag as HYPOTHESIS
2	Passes reproduction in clean environment?	Proceed to Step 3	Discard
3	Has valid evidence class label?	Proceed to Step 4	Request or Discard
4	Independent verification available?	Assign T2	Assign T1 only
5	Formal proof or peer-equivalent check?	Assign T3	Remains T2

Figure 1 (below) shows the decision gate as a flowchart. The gate handles both deterministic and stochastic outputs — see Section 6.3 for stochastic T2 criteria.

ZTEK Decision Gate — Flow
AI Output Generated	v [QUARANTINE STATE: T0]	Has reproduce command? --> NO --> DISCARD / flag HYPOTHESIS	YES Reproduction passes in clean env? --> NO --> DISCARD	YES Has evidence class label? --> NO --> Request label or DISCARD	YES v PROMOTE to T1 (Working Hypothesis)	Independent verification available? --> NO --> STAYS T1	YES v PROMOTE to T2 (Working Result)	Formal proof or peer-equivalent? --> NO --> STAYS T2	YES v PROMOTE to T3 (Certified Result) --> ENTERS KNOWLEDGE BASE

Figure 1. ZTEK Decision Gate flowchart. Each stage is a hard gate; claims cannot advance without satisfying the check at that stage.

6. Trust Tier System

The trust tier system provides the operational vocabulary for ZTEK. Every claim in the system has an explicit tier at all times.

Tier	Name	Criteria	Permitted Actions
T0	Raw Output	AI-generated, no verification	None. Quarantined.
T1	Hypothesis	Passed reproduction test	Informs analysis. Not citable.
T2	Working Result	Passed independent verification	Used in analysis. Internal citation.
T3	Certified Result	Formal proof or peer-equivalent	Full citation. Paper-ready.

6.1 Tier Transition Rules

- Tiers advance through verification only — never through assertion, fluency, or consensus
- Tiers reset on context change (Pillar 5)
- Tiers cannot be skipped
- Tier assignment is logged with timestamp, agent, and verification method

6.2 AEAL Integration

An agent with a degraded AEAL health score may have its effective T2 ceiling lowered to T1, requiring additional verification steps before promotion. In the current reference implementation, this ceiling is enforced through a human-in-the-loop policy: before any T2 promotion, the researcher checks the agent's AEAL health score for the current session. If the fabrication rate (FR) exceeds 0.2 or the capability honesty (CH) score falls below 0.6, T2 promotion is blocked until a second independent verification is completed. Automated enforcement via a trust tier API is planned as future work.

For organizational adoption guidance connecting AEAL longitudinal tracking to ZTEK transactional enforcement, see Paper 10 of this series.

6.3 Handling Stochastic Outputs

Not all AI outputs are deterministic. For stochastic outputs (e.g., sampling-based generation, non-seeded numerical searches), exact reproduction is not always achievable. ZTEK handles this through statistical T2 criteria:

- Run the reproduce command N >= 5 times with different seeds
- Compute the mean and standard deviation of the key numerical claim
- T2 is granted if: (a) the claimed value lies within 2 standard deviations of the mean across runs, and (b) the coefficient of variation is below 5%
- If variance is high, the claim is flagged as stochastic-T1 and requires additional independent verification before T2 promotion

For deterministic claims (fixed seed, exact reproduce command), exact match is required for T2.

7. Integration with AEAL

ZTEK and AEAL are complementary, not redundant. AEAL operates longitudinally — it tracks agent health, fabrication rate, and capability honesty over time. ZTEK operates at the transaction level — it decides whether a specific claim may cross the boundary right now.

Dimension	AEAL	ZTEK
Temporal scope	Longitudinal (session history)	Transactional (single claim)
Primary function	Agent health tracking	Boundary enforcement
Key metric	Fabrication Rate, Capability Honesty	Trust Tier, Provenance completeness
Failure response	Health score degradation	Claim rejection / quarantine
Output	Agent trust profile	Verified knowledge base entry

8. Implementation Guide

8.1 Minimum Viable Implementation

For teams adopting ZTEK incrementally, the minimum viable implementation consists of three rules:

- Rule 1: Any numerical constant must have a reproduce command. No command, no trust.
- Rule 2: Any structural claim must have an independent verification path — either a second computation or a known ground-truth case it correctly handles.
- Rule 3: Any claim that first appears in a relay chain is automatically flagged for re-derivation from first principles before entering the paper.

8.2 Full Implementation

Full ZTEK implementation requires:

- Quarantine tagging system for all AI outputs
- Provenance schema enforced at knowledge base entry (see Appendix A)
- Independent verification pipeline (separate environment, separate model where possible)
- Trust tier registry with audit log (see Appendix C)
- Context change detection triggering automatic tier reset

For organizational adoption, pilot design, and change management guidance, see Paper 10 of this series (Change Management Framework).

8.3 Operational Cost Estimates

Based on the PCF research program reference implementation, the following overhead estimates apply for a typical numerical claim verification cycle:

Activity	Median Time	Notes
Provenance field completion (per claim)	3–5 minutes	Mostly copy-paste from run log
Reproduce command execution (deterministic)	30 sec – 5 min	Depends on computation cost
Reproduce command execution (stochastic, N=5)	5–25 minutes	Parallelizable
Independent verification (second environment)	10–30 minutes	One-time env setup amortized
Audit log entry	1–2 minutes	Templated; see Appendix C
Full T0 → T2 cycle (typical claim)	15–45 minutes	Majority is verification, not paperwork

The overhead is approximately 20–30% additional time per scan cycle relative to unverified operation. In the PCF program pilot, this overhead was accepted as worthwhile after the first caught fabrication — a claim that had already been referenced in a draft section.

9. Empirical Validation

9.1 Methods

Retrospective validation was conducted by the human author against the five documented failure incidents in the PCF research program (Section 3). For each incident, the author applied

the ZTEK decision gate retroactively, asking: at which gate step would this claim have been stopped?

Validation was performed by the human author, not by an AI system, to avoid self-assessment bias. Each incident was evaluated against the gate independently, without reference to prior incident evaluations. The evaluate-and-record procedure was: (1) reconstruct the failure path from research logs, (2) identify the earliest gate step the claim would have failed, (3) record the counterfactual outcome.

Research environment: VS Code workspace on Windows, Python 3.11, mpmath 1.3.0, GitHub Copilot (GPT-4 backend), Claude 3 Opus. Logs maintained in Git repository at github.com/papanokechi/pcf-research.

9.2 Results

Incident	Gate Step Failed	Would ZTEK Have Prevented It?	Mechanism
Dual-path fabrication	Step 1 (no reproduce command)	Yes	Pillar 1 quarantine stops propagation at first boundary
PSLQ trivial relation flaw	Step 2 (reproduction fails with rationality filter)	Yes	Pillar 2 provenance check catches missing filter
Relay chain propagation	Step 3 (no evidence class; inherited from prior model)	Yes	Pillar 3 boundary reset breaks inheritance chain
Searcher's Fatigue	Step 4 (T0 promoted to T3 without independent verification)	Partially	Pillar 4 tier enforcement slows but does not eliminate the pattern without organizational measures (see Paper 10)
Catalog invalidation	Pillar 5 (context change; no tier reset)	Yes	Pillar 5 context reset triggers re-verification

Result: 4 of 5 incidents would have been fully prevented. The remaining incident (Searcher's Fatigue) reflects a behavioral pattern requiring complementary organizational change — addressed in Paper 10 of this series.

10. Limitations and Future Work

10.1 Limitations

- ZTEK addresses structural fabrication, not all AI errors. Errors within verified computations (e.g., numerical precision issues) require domain-specific verification beyond the protocol scope.
- ZTEK addresses structural fabrication arising from statistical continuation pressure, not adversarial manipulation or poisoned training data. Integrating adversarial-robustness checks is future work.
- The trust tier system assumes reproducibility. For stochastic outputs, T2 verification requires statistical rather than exact reproduction (see Section 6.3 for criteria).
- Pillar 5 re-verification can be operationally demanding in fast-moving research workflows. The minimum viable implementation rules (Section 8.1) are provided as a practical alternative.
- Retrospective validation was performed by the human author on their own incident record. Independent third-party validation of the protocol on external datasets is a priority for future work.

10.2 Future Work

- Automated provenance tagging via IDE integration (VS Code extension planned) — this is the highest-leverage automation opportunity; the reproduce command and environment fingerprint can be captured programmatically at generation time
- Trust tier API for integration with paper management systems
- Cross-organization failure mode registry for community-level learning
- Third-party prospective validation study on a separate research program
- Formal complexity analysis of verification overhead vs. fabrication risk reduction

11. Conclusion

ZTEK provides a practical, grounded answer to the central challenge of AI-assisted knowledge work: how do we reliably distinguish what the AI generates from what we have verified?

The answer is architectural, not behavioral. We do not ask the AI to be more careful. We build a boundary it cannot cross without earning passage.

The five pillars — assume fabrication, require provenance, verify at boundary, enforce least confidence, re-verify on context change — are not theoretical constructs. They are direct responses to documented failures in real research workflows. The protocol is, in this sense, empirically derived.

Together with AI Behavior Science (Paper 7) and AEAL (Paper 8), ZTEK completes the enforcement layer of the AI Epistemic Governance Stack. The organizational adoption layer is addressed in Paper 10.

References

[1] AI Behavior Science — Founding Territory Paper. Zenodo. DOI: 10.5281/zenodo.19562751

[2] AEAL: Agent Epistemic Accountability Layer. Zenodo. DOI: 10.5281/zenodo.19565086

[3] Rose, J. et al. Zero Trust Architecture. NIST Special Publication 800-207. National Institute of Standards and Technology. 2020.

[4] Shumailov, I. et al. The Curse of Recursion: Training on Generated Data Makes Models Forget. arXiv:2305.17493. 2023.

[5] Ji, Z. et al. Survey of Hallucination in Natural Language Generation. ACM Computing Surveys, 55(12), 1–38. 2023. https://doi.org/10.1145/3571730

AI Disclosure:
This paper was developed in collaboration with Claude (Anthropic) as a strategic research partner. All claims, failure mode documentation, and protocol design were verified by the human author. ZTEK was developed to address real failure modes encountered in the author's own research workflow, with AI assistance used for structuring, drafting, and critique.

Appendix A — Provenance JSON Schema

The following JSON schema defines the required provenance record for any claim entering the ZTEK-protected knowledge base. All fields are required except source_chain (required for relay-chain outputs) and stochastic_stats (required for stochastic outputs only).

provenance_schema.json

```
{
  "claim_id":          "pcf-2024-0047",
  "claim_text":        "V_quad = GCF(1, 3n^2+n+1) is irrational",
  "evidence_class":    "independently_verified",
  "reproduce_command": "python verify_vquad.py --digits 2200 --seed 42",
  "environment": {
    "python":  "3.11.4",
    "mpmath":  "1.3.0",
    "commit":  "a3f92c1",
    "os":      "Windows 11 / Ubuntu 24 (WSL2)"
  },
  "source_chain": [
    "Claude-3-opus (session 2024-11-15)",
    "Copilot-gpt4 (session 2024-11-15)",
    "human-review (papanokechi)"
  ],
  "timestamp":  "2024-11-15T09:34:12Z",
  "tier":       "T2",
  "tier_history": [
    { "tier": "T0", "timestamp": "2024-11-15T08:10:00Z", "actor": "Claude-3-
opus" },
    { "tier": "T1", "timestamp": "2024-11-15T09:00:00Z", "actor": "human-review",
      "method": "reproduce_command_passed" },
    { "tier": "T2", "timestamp": "2024-11-15T09:34:12Z", "actor": "human-review",
      "method": "independent_mpmath_verification_2200_digits" }
  ],
  "stochastic_stats": null
}
```

For stochastic outputs, replace the stochastic_stats field with:

stochastic_stats field (stochastic outputs only)

```
  "stochastic_stats": {
    "n_runs":    5,
    "seeds":     [42, 43, 44, 45, 46],
    "mean":      1.197371264,
    "std_dev":   0.000000003,
    "cv_pct":    0.00000025,
    "t2_passed": true
  }
```

Appendix B — Sample Reproduce Commands

The following reproduce commands are taken from the PCF research program. They illustrate the level of specificity required for a valid ZTEK reproduce command.

B.1 — Numerical constant verification (deterministic)

verify_vquad.py invocation

```
# Verify V_quad = GCF(1, 3n^2+n+1) to 2200 digits
# Environment: Python 3.11.4, mpmath 1.3.0, commit a3f92c1
python verify_vquad.py --digits 2200 --seed 42 --output vquad_result.txt

# Expected output (first 40 digits):
# V_quad = 1.197371264535370517169803743578...
# Status: PASS | evidence_class: independently_verified
```

B.2 — PSLQ relation check with rationality pre-filter (post-fix)

pslq_check.py invocation

```
# Check candidate relation against rationality pre-filter
# This command implements the fix for the degree-2 trivial relation flaw
python pslq_check.py \
  --constant V_quad \
  --degree 3 \
  --rationality-prefilter enabled \
  --digits 500 \
  --commit a3f92c1

# Rationality pre-filter: rejects relations where all terms
# are rational multiples of each other (degree-2 trivial).
# Status: PASS | evidence_class: numerical_identity
```

B.3 — Lean 4 formal proof verification

Lean 4 verification

```
# Verify Casoratian identity for pi/4 series
# Requires: Lean 4.3.0, Mathlib commit b7a3c2f
lake build CasoratianPi4

# Expected output:
# Build completed successfully.
# Theorems verified: 12  |  Sorry stubs: 0
# evidence_class: formalized
```

Appendix C — Sample Audit Log Entry

The following shows a complete audit log entry for a claim that passed from T0 to T2. The audit log is maintained as a newline-delimited JSON file (claims_audit.ndjson) in the research repository.

claims_audit.ndjson — single entry

```
{
  "event":       "tier_transition",
  "claim_id":    "pcf-2024-0047",
  "from_tier":   "T1",
  "to_tier":     "T2",
  "timestamp":   "2024-11-15T09:34:12Z",
  "actor":       "human-review",
  "agent":       "Claude-3-opus",
  "method":      "independent_mpmath_verification",
  "details": {
    "verification_digits": 2200,
    "reference_value":  "1.197371264535370517169803743578",
    "computed_value":   "1.197371264535370517169803743578",
    "match":            true,
    "environment":      "Ubuntu 24 WSL2 / Python 3.11.4 / mpmath 1.3.0"
  },
  "notes": "Independent verification in clean WSL2 environment.
            Matches Claude session result exactly to 40 significant digits.",
  "context_hash": "sha256:9f3a1b..."
}
```

The context_hash field is a SHA-256 hash of the full session context at time of tier transition. This field enables Pillar 5 (re-verify on context change): if the context hash changes, the claim is automatically flagged for re-verification before its tier is accepted in the new context.